

AiRWeb: Using AR to Extend Web Browsing Beyond Handheld Screens

Mengfei Gao

ILDA

Université Paris-Saclay, CNRS, Inria
Gif-sur-Yvette, France
mengfei.gao@universite-paris-saclay.fr

Ludovic David

ILDA

Université Paris-Saclay, CNRS, Inria
Gif-sur-Yvette, France
ludovic.david@inria.fr

Caroline Appert

ILDA

Université Paris-Saclay, CNRS, Inria
Gif-sur-Yvette, France
caroline.appert@universite-paris-saclay.fr

Emmanuel Pietriga

ILDA

Université Paris-Saclay, CNRS, Inria
Gif-sur-Yvette, France
emmanuel.pietriga@inria.fr

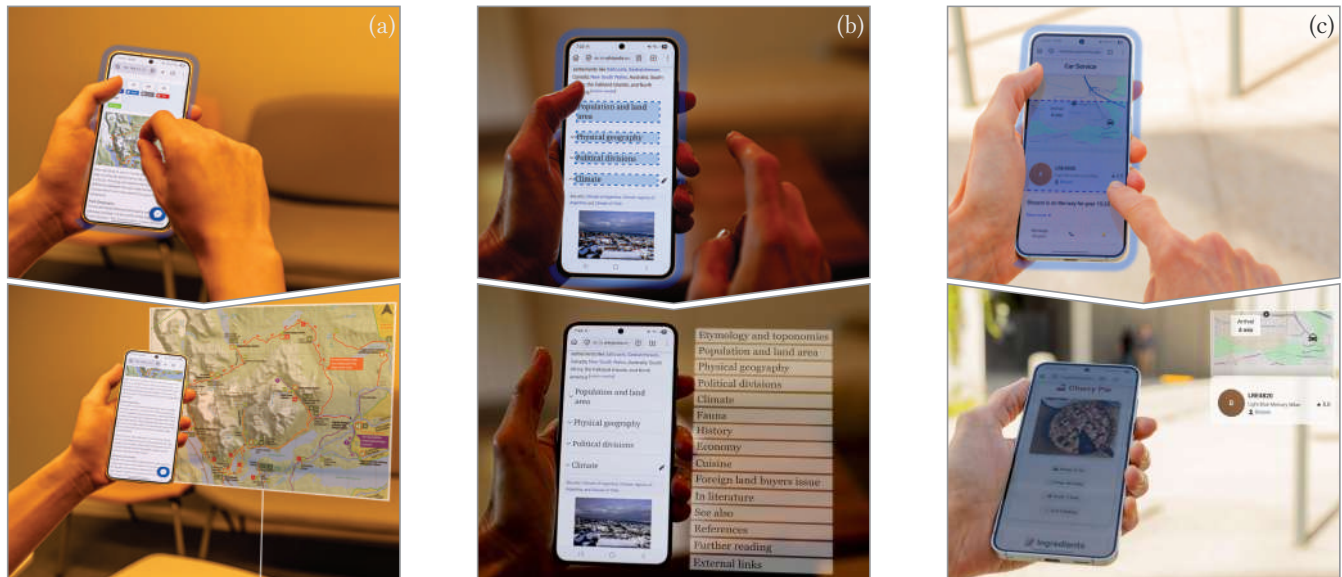


Figure 1: Examples of Web page elements moved beyond the screen: (a) offloading a map above the coffee table so that it remains visible when scrolling down the page; (b) offloading all section headings of a document at once to get an interactive outline; (c) offloading a rubber-band selection to monitor rendez-vous with a car pickup while reading about a cooking recipe.

Abstract

Browsing the Web on mobile devices is often cumbersome due to their limited screen space. We investigate a phone+AR Web browsing approach, AiRWeb, that leverages the structural properties of Web pages to allow users to seamlessly select and offload arbitrary Web content into the space surrounding them. Focusing on flexibility, AiRWeb lets users decide what to offload, when to do so, and how offloaded content is arranged, enabling personalized

organization tailored to the task at hand. We developed a fully functional prototype using standard Web technologies, that covers the complete interaction workflow, from the selection of elements to offload from the phone to their manipulation in the air. Results from a preliminary study conducted using this prototype suggest that AiRWeb is learnable and usable, while also revealing open design challenges around offload mode activation in particular.

CCS Concepts

• Human-centered computing → Interaction techniques; Mixed / augmented reality.

Keywords

Augmented Reality, Web browsing, Cross-Device Interaction



This work is licensed under a Creative Commons Attribution 4.0 International License.
CHI EA '26, Barcelona, Spain

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2281-3/26/04
<https://doi.org/10.1145/3772363.3798342>

ACM Reference Format:

Mengfei Gao, Caroline Appert, Ludovic David, and Emmanuel Pietriga. 2026. AiRWeb: Using AR to Extend Web Browsing Beyond Handheld Screens. In *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA '26)*, April 13–17, 2026, Barcelona, Spain. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3772363.3798342>

1 Introduction

Mobile Web browsing is constrained by the limited display area of handheld devices. As Augmented Reality (AR) technologies mature, handheld devices can be used in tandem with AR eyewear to increase effective screen real estate and extend interaction beyond the physical phone [10]. This combination can enhance the mobile Web browsing experience, which often involves navigating long pages and repeatedly jumping back and forth between dispersed pieces of content. Given that AR displays in mobile Web contexts are a novel interaction space, it is difficult to anticipate how users will appropriate them to organize content. Designing systems that enable flexible, user-driven offloading of Web elements provides an opportunity to support a wide range of individual strategies, without constraining users to predefined patterns.

Table 1 provides an overview of key prior work on phone+AR displays, organized along two design dimensions:

- **Offloading control** specifies who decides which content moves to the AR display. Some systems do not give end-users control over this: Beyond-the-Phone [11] relies on designer-defined application views that can be transitioned to AR; WebJump [9] requires developers to annotate webpage elements for offloading. In contrast, Push2AR [7] allows end-users to directly offload list elements using swipe gestures.
- **Offloading scope** captures the granularity of content that can be offloaded. Beyond-the-Phone [11] focuses on monolithic application views. Push2AR [7] supports offloading individual items from flat lists but does not support other types of Web elements.

Existing approaches limit users' ability to decide what, when, and where to offload content from the phone to the air. To address this gap, we introduce AiRWeb, a mobile Web browsing prototype that allows users to offload arbitrary parts of Web pages into AR and spatially arrange them around the user (Figure 1). Unlike systems that rely on designer-defined views or developer-authored annotations, AiRWeb operates on regular Web content, leveraging the generic DOM structure of Web pages to enable flexible, user-driven selection and offloading.

We contribute the first complete interaction design and implementation of a phone-based, AR-enhanced Web browsing system, from the selection of Web page elements to offload from the phone to their interactive manipulation in AR. We present the findings from a preliminary study performed using this prototype, that provides initial evidence about the relevance and usability of the approach, and also highlights open design challenges.

2 Motivating Scenario

AiRWeb is designed to enable users to reorganize elements of arbitrary web pages in the space around their handheld device, supporting more flexible and efficient mobile web browsing. By allowing

users to offload selected elements into AR, AiRWeb helps reduce repetitive navigation actions and enables side-by-side access to relevant information, as we illustrate in the following short scenario.

While waiting for an appointment, Noa starts planning for an upcoming hiking trip. They browse a Web page describing two alternative routes, that includes descriptions and a map showing both of them. To keep the map visible while the continue reading, Noa activates offloading mode and selects *the map alone*, placing a larger version *above the coffee table in front of them* (Figure 1-a). They can then keep scrolling and reading while referring to the map at will.

Later, Noa wants to learn more about Patagonia, the region in which they will be hiking. They go to Wikipedia on their phone. The page describing Patagonia is pretty long and features many sections. Noa is just browsing serendipitously, curious to discover interesting things about the region. They offload *all section headers at once near the phone* using a single gesture. They can now use the resulting document outline to jump between sections on the phone (Figure 1-b).

After their appointment, Noa requests a ride home. While waiting for the car to arrive, they begin searching for a cherry pie recipe for guests arriving later that evening. As they want to keep track of the car's location and estimated time of arrival, they perform a continuous selection of the *relevant area* of the ride service page that shows the map and time information, and offload it into *a corner of their field of view* (Figure 1-c). This allows them to continue browsing recipes while monitoring the ride information without switching back and forth between Web pages.

This scenario illustrates the need for flexible **selection at different granularities**: *single elements*, *groups of elements*, and *arbitrary areas* of the page; and **placement options**: *near the phone*, *fixed in the field of view*, or *anchored in the physical world*.

3 AiRWeb

We conducted a formative elicitation study (N=13) to investigate users' expectations (UE) for offloading interactions. Participants proposed techniques for transferring elements from phone to AR, explored placement strategies, and described ways to discard offloaded content. From this study, we derived four key insights:

- **UE₁: Activation.** Offloading gestures typically started on or near the phone and ended in mid-air.
- **UE₂: Selection.** Single-point selection was most common, with participants expecting the system to resolve ambiguities between nested or nearby elements.

Table 1: Positioning of representative phone+AR systems along two dimensions: offloading control (who determines what content is moved to AR) and offloading scope (the granularity of offloaded content).

System	Offloading control	Offloading scope
Push2AR [7]	User-driven	List items
Beyond-the-Phone [11]	Designer-defined	Monolithic application views
WebJump [9]	Developer-defined	Arbitrary Web elements
AiRWeb	User-driven	Arbitrary Web elements

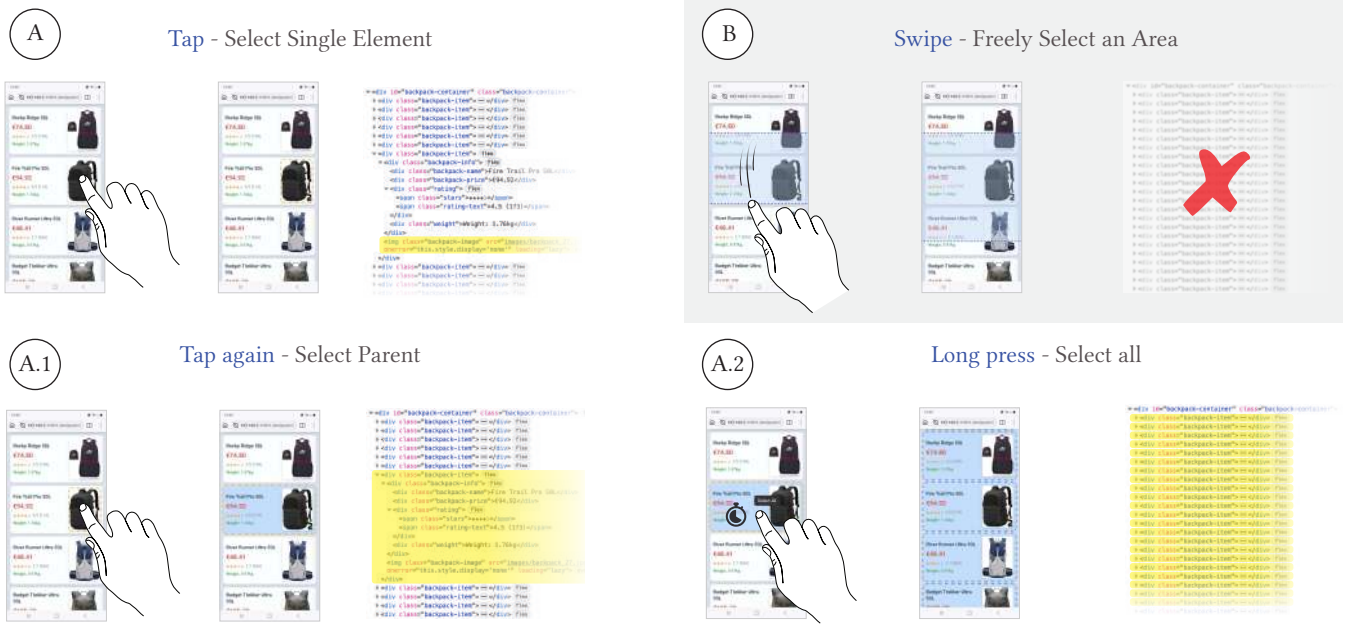


Figure 2: Different types of element selections: (A) Tapping on an element selects it; (B) Performing a vertical drag makes a rubberband selection that can freely span elements regardless of the DOM hierarchy.; (A.1) Tapping again on an already-selected element expands the selection to its parent in the DOM hierarchy; and (A.2) Performing a long-press on an already-selected element pops up a button to expand the selection to all similar elements in the DOM hierarchy.

- **UE₃: Placement.** Anchoring of offloaded content was usually determined by the gesture’s endpoint.
- **UE₄: Removal.** Participants used gestures such as throwing the element away or returning it to the phone.

Activation. Participants used a regular mobile Web browser featuring the default, primarily touch-based, interactive capabilities. They were also asked to avoid conflicts with those capabilities when making proposals for offloading commands. Despite this, participants often proposed touch interactions that conflicted with those default capabilities.

To allow users to rely on simple gestures without conflicting with touch interactions, we introduced an explicit offloading quasi-mode. Users enter the mode by touching the side of the screen with the thumb holding the phone. A thumb silhouette appears in AR near the phone to indicate the action, and a blue halo signals that offloading mode is active, as illustrated in the top row of Figure 1. Those offloading quasimode exits automatically when lifting the thumb, returning the system to default browsing mode.

Selection. With offloading mode active, content can be transferred via a *finger-swipe offload* (touch+air) or an *air-pinch offload* (air-only). During quick interactions, the system infers the target element (UE₂) by traversing the DOM to identify the first CSS block-level element at the gesture’s start point (e.g., a `<p>` or a `<div>` element).

For more granularity in selection control (Figure 2), users can **tap to select** the lowest-level element at that position, incrementally expanding the selection to parent elements; or they can long-press

to select a collection of similar elements with a single action. Alternatively, users can **drag to select** an arbitrary rectangular region. During selection, AirWeb provides immediate visual feedback by modifying the CSS styling of selected DOM elements, clearly indicating what will be offloaded.

Management (Placement and Removal). When using a *finger-swipe offload*, the selection is by default offloaded near the phone and remains anchored to it. During an *air-pinch* offload, the final anchoring depends on the gesture’s endpoint (UE₃). As shown in Figure 3, the hand typically traverses *Region_{Phone}* first. An orange plane aligned with the phone is shown, indicating that the element being offloaded will be anchored to the phone if released now. Otherwise, if continuing to *Region_{World}*, AirWeb provides feedback via a vertical line anchoring the element to the nearest horizontal surface (e.g., the floor or a table), decorating the element with a 3D frame to suggest its anchoring into the physical environment. If continuing further, bring the element closer to the user’s face, the hand enters *Region_{Fov}*. The vertical anchoring line and 3D frame disappear, and a blue plane signals fixed anchoring in the user’s field of view.

Elements anchored to the phone remain fixed but can be repositioned via pinch-and-drag. They can be toggled to a scrollable layout reflecting their original order. Offloaded items remain linked to the phone, acting as bookmarks and shortcuts: tapping an offloaded item scrolls the mobile page to show its counterpart, similar to what Push2AR does for list items [7]. Users can discard offloaded elements by throwing them away (UE₄).

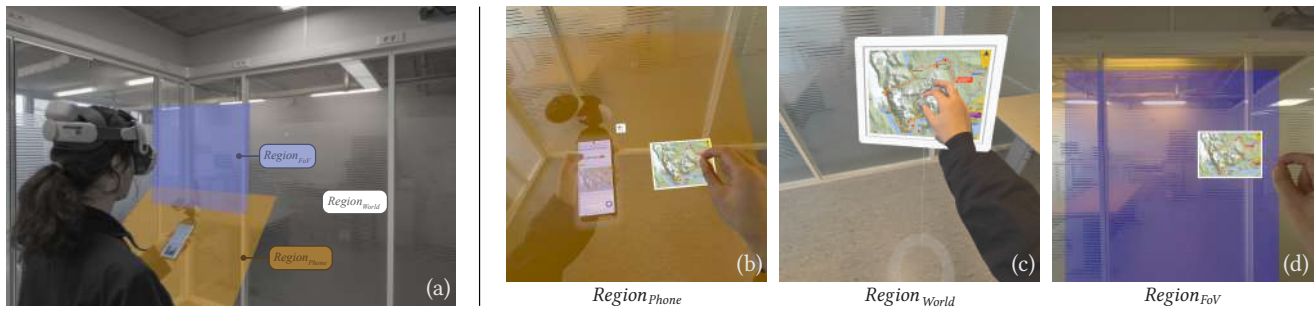


Figure 3: AiRWeb partitions space into three regions: $Region_{phone}$ (orange), $Region_{FoV}$ (blue), and $Region_{World}$. Anchoring depends on the region where the element is released. Visual feedback guides users by revealing planar regions or highlighting the element’s relationship to physical surfaces.

4 Implementation

AiRWeb is a fully functional prototype built entirely with Web technologies. On the smartphone, it runs as a Manifest V3 browser extension implemented in JavaScript. One injected script handles core functionalities such as gesture detection, element selection, and feedforward for offloading. A second background script renders selected elements to offscreen buffers, ensuring visual consistency with the original page. This is important because the mobile browser and the WebXR client on the AR headset differ in screen dimensions, aspect ratios, fonts, and rendering behavior, which would necessarily introduce appearance differences if the renderings were performed independently. Instead, elements are captured on the phone and pre-transmitted before the offload gesture, reducing latency and preserving fidelity.

On the AR headset, a WebXR client built with A-Frame and Three.js receives the offloaded elements, tracks users’ hands for mid-air interactions, and monitors the smartphone’s position to anchor elements in $Region_{phone}$. For precise, low-latency tracking, a motion controller is attached to the smartphone, providing performance comparable to external tracking systems without requiring a fixed setup. A NodeJS server handles communication between devices using Socket.IO. The system has been evaluated on a Meta Quest 3 headset and a Samsung Galaxy S10 running Firefox nightly.

5 Preliminary Evaluation

We conducted a user study ($N=12$, aged 22–37) with researchers in HCI/Visualization to get preliminary feedback about AiRWeb’s usability and practical utility. The study comprised a *discovery phase*, where participants explored ten features to assess discoverability; and a *scenarios phase*, where they completed five contextualized tasks adapted from our motivating scenarios (Section 2).

Discovery Phase. Participants explored each feature at their own pace, consulting a help video if needed. Average exploration time was 32 minutes (median 23 minutes) for all ten features, always presented in the same order, covering the full interaction workflow and selection/placement options. As shown in Figure 4, participants sought more help by watching videos in two cases: to offload elements, and to discard them (2nd and 3rd rows, respectively). This was due to misunderstandings about quasimode activation

and discard gestures. Participants did not seek as much help about subsequent features, as they learnt from prior exploration steps.

Scenarios Phase. Observations in this phase suggest an effective and personalized appropriation of AiRWeb. All participants successfully completed the 5-step scenario. We also observed variations in how participants selected and offloaded elements. Eight participants consistently used pinch gestures to offload a selection, while two participants always relied on flick gestures. When scenarios required selecting nested elements (e.g., A.2 in Figure 2), two-thirds of participants used multiple taps (*expand selection to enclosing elements*), whereas one-third preferred using drag gestures to select a rectangular area that enclosed the target element (*select an arbitrary rectangular region*).

User Experience. Participants completed the *User Experience Questionnaire (UEQ)*¹ at the end of the study. Ratings were positive across all dimensions: Attractiveness ($M=2.13$), Stimulation ($M=2.27$), Novelty ($M=2.04$), Efficiency ($M=1.96$) were *excellent*, reflecting engagement and usefulness. Perspicuity ($M=1.38$) and Dependability ($M=1.48$) were above average, with some challenges reported for discard gestures and quasimode activation.

6 Discussion

Prior work has explored distributing Web interfaces across physical devices to overcome problems of limited screen space, either through developer-oriented frameworks [2, 5, 8] or systems that allow users to redistribute elements across multiple displays at runtime [3, 4, 6]. These approaches typically focus on specific applications, predefined device ecologies, or semi-automatic adaptations, often in collaborative or task-specific settings.

AiRWeb targets single-user, mobile Web browsing activities and investigates how Augmented Reality can enhance them by enabling users to offload selected page elements around their smartphone. It emphasizes user-driven, on-the-fly control over selection and placement of arbitrary elements from rich, structured Web pages without requiring application-specific support, predefined layouts, or adaptation rules. Following a phone-centric approach [10], AiRWeb uses AR to extend the smartphone rather than replace it. Our fully-functional prototype, built entirely on existing Web technologies, demonstrates that carefully designed interaction can make

¹<https://www.ueq-online.org>

Activate AiRWeb mode -	✓	▶	✓	✓	▶	✓	✓	▶	✓	✓	✓	▶
Offload element around the phone -	▶	▶	▶	✓	▶	✓	▶	✓	✓	▶	▶	✓
Remove offloaded elements -	▶	▶	✓	✓	✓	▶	▶	▶	▶	▶	▶	▶
Offload element to world / field-of-view -	✓	▶	✓	✓	✓	✓	✓	▶	✓	✓	▶	▶
Move offloaded element to another position -	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Toggle elements scroll behavior (fixed vs. scroll-linked) -	✓	▶	✓	✓	✓	✓	✓	✓	✓	✓	▶	✓
Expand a selection to its higher-level element -	✓	✓	✓	✓	✓	✓	✓	▶	✓	✓	▶	✓
Select and offload a group of similar elements -	✓	▶	✓	✓	▶	✓	✓	✓	✓	▶	▶	▶
Select and offload an arbitrary rectangular region -	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	▶	✓
Navigate back to original location of offloaded element -	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	1	2	3	4	5	6	7	8	9	10	11	12
	Participant											

Figure 4: Help video consultations by feature × participant. Each cell indicates whether a participant (column) consulted the help video for the feature (row): ✓ = no video consulted, ▶ = video consulted.

user-driven spatial offloading of live Web content feasible today. As AR eyewear continues to miniaturize and improve, such approaches could become increasingly integrated into everyday mobile browsing activities, once the following challenges are addressed.

Interaction Design Challenges. The study highlighted several interaction design challenges. First, participants sometimes performed the discard gesture in the opposite direction than intended, reflecting ambiguity between two metaphors: either “throwing away” or “putting back inside the phone”. We plan to explicitly support both. Second, visual feedforward for the offloading quasimode was misinterpreted by some participants as an instruction to press a physical button on the phone, underscoring the difficulty of conveying subtle, one-handed gestures across devices. We are exploring alternative quasimode activations, including dedicated hardware buttons or AR-mounted virtual buttons with vision-based detection of thumb contact, which will become more feasible as hand-object interaction tracking improves.

System and Architectural Challenges. Beyond questions about interaction design, the implementation of AiRWeb revealed system-level challenges. Its emphasis on high-fidelity visual consistency between phone and AR renderings limits offloaded elements to static snapshots for now. Supporting dynamic content while maintaining visual consistency would likely require streaming approaches (e.g., via WebRTC), raising issues of performance, latency, and network bandwidth overuse. Similarly, interaction with offloaded content is not yet supported, highlighting the need for tighter coupling between input, state, and execution contexts across mobile browsers and AR environments. We are also working toward updating offloaded elements from background tabs. Combined with interactive offloaded content, this would better support parallel Web browsing, a task that is particularly challenging on mobile devices [1].

References

- [1] Khalad Hasan, David Ahlström, Junhyeok Kim, and Pourang Irani. 2017. AirPanels: Two-Handed Around-Device Interaction for Pane Switching on Smartphones. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems* (Denver, Colorado, USA) (CHI '17). Association for Computing Machinery, New York, NY, USA, 679–691. doi:10.1145/3025453.3026029
- [2] Clemens N. Klokmoose, James R. Eagan, Siemen Baader, Wendy Mackay, and Michel Beaudouin-Lafon. 2015. Webstrates: Shareable Dynamic Media. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology* (Charlotte, NC, USA) (UIST '15). Association for Computing Machinery, New York, NY, USA, 280–290. doi:10.1145/2807442.2807446
- [3] Michael Nebeling. 2017. XDBrowser 2.0: Semi-Automatic Generation of Cross-Device Interfaces. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems* (Denver, Colorado, USA) (CHI '17). Association for Computing Machinery, New York, NY, USA, 4574–4584. doi:10.1145/3025453.3025547
- [4] Michael Nebeling and Anind K. Dey. 2016. XDBrowser: User-Defined Cross-Device Web Page Designs. In *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems* (San Jose, California, USA) (CHI '16). Association for Computing Machinery, New York, NY, USA, 5494–5505. doi:10.1145/2858036.2858048
- [5] Michael Nebeling, Theano Mints, Maria Husmann, and Moira Norrie. 2014. Interactive development of cross-device user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Toronto, Ontario, Canada) (CHI '14). Association for Computing Machinery, New York, NY, USA, 2793–2802. doi:10.1145/2556288.2556980
- [6] Seonghoon Park, Jeho Lee, Yonghun Choi, and Hojung Cha. 2024. Vulture: Cross-Device Web Experience with Fine-Grained Graphical User Interface Distribution. In *IEEE INFOCOM 2024 - IEEE Conference on Computer Communications*. 2478–2487. doi:10.1109/INFOCOM52122.2024.10621433
- [7] Jonathan Wieland, Hyunsung Cho, Sebastian Hubenschmid, Akihiro Kiuchi, Harald Reiterer, and David Lindlbauer. 2024. Push2AR: Enhancing Mobile List Interactions Using Augmented Reality. In *2024 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. 671–680. doi:10.1109/ISMAR62088.2024.00082
- [8] Jishuo Yang and Daniel Wigdor. 2014. Panelrama: enabling easy specification of cross-device web applications. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (Toronto, Ontario, Canada) (CHI '14). Association for Computing Machinery, New York, NY, USA, 2783–2792. doi:10.1145/2556288.2557199
- [9] Xin Zeng, Xiaoyu Wang, Zhengtai Gou, Yiqiang Chen, and Tengxiang Zhang. 2023. WebJump: AR-facilitated Distributed Display of Web Pages. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) (CHI EA '23). Association for Computing Machinery, New York, NY, USA, Article 317, 6 pages. doi:10.1145/3544549.3585669
- [10] Fengyuan Zhu and Tovi Grossman. 2020. BISHARE: Exploring Bidirectional Interactions Between Smartphones and Head-Mounted Augmented Reality. In *Proceedings of the 2020 CHI Conference on Human Factors in Computing Systems* (CHI '20). ACM, 1–14. doi:10.1145/3313831.3376233
- [11] Fengyuan Zhu, Xun Qian, Daniel Kalmar, Mahdi Tayarani, Eric J. Gonzalez, Mar Gonzalez-Franco, David Kim, and Ruofei Du. 2025. Beyond the Phone: Exploring Phone-XR Integration through Multi-View Transitions for Real-World Applications. In *2025 IEEE Conference Virtual Reality and 3D User Interfaces (VR)*. 770–780. doi:10.1109/VR59515.2025.00099